

وثيقة شرح النظام

System Overview Document

منصة نَسَق | NASSAQ

1) التعريف العام بالنظام

نَسَق | NASSAQ هو School Operating System متكامل، تم تصميمه ليكون منصة تشغيل مركزية لإدارة المدارس وتشغيلها رقميًا من خلال بيئة موحدة تربط جميع العمليات الأكاديمية والإدارية والتشغيلية داخل نظام واحد.

النظام ليس مجرد لوحة تحكم أو تطبيق لإدارة المستخدمين، وليس أيضًا مجرد نظام لإدخال بيانات الطلاب أو الجداول، بل هو منظومة تشغيل مدرسية كاملة تهدف إلى إدارة المدرسة ككيان حي يعمل يوميًا عبر مجموعة مترابطة من المحركات والكيانات والصلاحيات والواجهات.

الهدف من نَسَق هو تحويل المدرسة من كيان يعمل عبر ملفات متفرقة، وإجراءات يدوية، وأنظمة منفصلة، إلى منصة تشغيل ذكية ومترابطة تدير:

- المدارس
- المستخدمين
- الأدوار
- الفصول
- الطلاب
- المعلمين
- أولياء الأمور
- الجداول الدراسية
- الحضور والانصراف
- التقييمات
- الإشعارات
- التقارير
- التحليلات
- الإعدادات
- السجلات
- السياسات العامة

وذلك من خلال قاعدة بيانات حقيقية تمثل المصدر الأساسي للحقيقة داخل النظام.

2) الفكرة الجوهرية للنظام

الفكرة الأساسية في نَسَق هي أن المدرسة لا تُدار من خلال صفحات منفصلة أو بيانات مبعثرة، بل من خلال نظام تشغيل موحد يتعامل مع جميع عناصر المدرسة ككيانات مترابطة.

بمعنى آخر:

- الطالب ليس مجرد اسم داخل جدول
- المعلم ليس مجرد حساب تسجيل دخول
- الفصل ليس مجرد بطاقة عرض
- الجدول ليس مجرد جدول مرسوم في الواجهة
- الإشعار ليس مجرد نص يظهر للمستخدم
- التقرير ليس رقمًا ثابتًا داخل Dashboard

بل كل عنصر في النظام يجب أن يكون:

- كيانًا حقيقيًا
- مخزنًا في قاعدة البيانات
- مرتبطًا بكيانات أخرى
- يؤثر ويتأثر بباقي النظام

3) طبيعة النظام: School Operating System

عندما نقول إن نَسَقُ هو School Operating System فهذا يعني أنه يعمل ك نظام تشغيل للمدرسة، وليس فقط ك School Management App.

الفرق أن التطبيق التقليدي قد يدير جزءًا من العمل، أما نَسَقُ فيجب أن يدير التشغيل الكامل للمدرسة، بما يشمل:

- إدارة الكيانات الأساسية
- إدارة العلاقات بين المستخدمين
- تنظيم التشغيل الأكاديمي
- تنظيم التشغيل اليومي
- تسجيل الأحداث
- قياس الأداء
- إصدار التنبيهات
- تحليل البيانات
- تمكين الإدارة المركزية والإدارة المحلية في نفس الوقت

أي أن النظام يجب أن يكون Operational, Transactional, Analytical, and Administrative في آنٍ واحد.

4) النظام متعدد المدارس

Multi-Tenant Architecture

نَسَقُ مبني على مفهوم Multi-Tenant Platform.

وهذا يعني أن المنصة الواحدة يمكنها تشغيل وإدارة عدة مدارس مستقلة داخل نفس النظام، مع الحفاظ الكامل على عزل البيانات بين كل مدرسة وأخرى.

معنى Multi-Tenant في نَسَقُ

كل مدرسة تعتبر Tenant مستقل داخل النظام، ولها:

- بياناتها الخاصة

- مستخدموها
- فصولها
- طلابها
- معلموها
- إعداداتها
- جداولها
- حضورها
- تقييماتها
- إشعاراتها
- تقاريرها

وفي الوقت نفسه، توجد طبقة أعلى هي Platform Level تدير المنصة بالكامل وتشرف على جميع المدارس من منظور مركزي.

أهمية هذا التصميم

هذا التصميم يحقق ما يلي:

- إمكانية تشغيل عدد كبير من المدارس داخل نفس المنصة
- تقليل تكلفة إدارة البنية التقنية
- توفير إدارة مركزية موحدة
- ضمان عزل البيانات بين المدارس
- دعم التوسع المستقبلي بسهولة

قاعدة أساسية

رغم أن النظام متعدد المدارس، فإن:

- مدير مدرسة لا يرى إلا بيانات مدرسته
- المعلم لا يرى إلا البيانات المصرح بها داخل سياق مدرسته ودوره
- المستخدم لا يستطيع الوصول إلى بيانات Tenant آخر
- كل استعلام وكل API وكل Dashboard يجب أن يحترم Tenant Context

5) الهوية الموحدة للمستخدم

Unified User Identity

يعتمد نَسَق على مفهوم Unified User Identity.

وهذا يعني أن كل مستخدم داخل النظام يمتلك هوية موحدة واحدة تمثل الأصل الأساسي له داخل المنصة، ويمكن ربط هذه الهوية بأكثر من دور إذا لزم الأمر.

ما معنى هذا عمليًا؟

قد يكون نفس الشخص:

- مدير منصة
- ومدير مدرسة
- وولي أمر

- أو معلم
- بحسب طبيعة استخدامه للنظام

بدل إنشاء حسابات منفصلة غير مترابطة، يتم إنشاء:

- User Identity واحدة
- ثم ربطها بـ Roles متعددة

فوائد هذا النهج

- تقليل التكرار
- منع ازدواجية الحسابات
- توحيد بيانات المستخدم
- دعم ميزة تبديل المستخدم / Switch Role
- تسهيل إدارة الصلاحيات
- الحفاظ على تاريخ المستخدم ونشاطه تحت هوية موحدة

قواعد مهمة

- المستخدم لا يملك نفس الدور مرتين
- كل Role يرتبط بـ Context واضح
- بعض الأدوار مرتبطة بـ Tenant / School
- وبعضها يكون على مستوى المنصة
- التبديل بين الأدوار يجب أن يغير:
 - الصلاحيات
 - الصفحات
 - البيانات المعروضة
 - القائمة الجانبية
 - السياق التشغيلي بالكامل

6) مستويان رئيسيان داخل النظام

Platform Level & School Level

نَسَقُ مبني على مستويين واضحين من التشغيل:

أولاً: مستوى المنصة

Platform Level

هذا هو المستوى الأعلى داخل النظام، ويخص الإدارة المركزية للمنصة.

من يعمل على هذا المستوى؟

- مدير النظام
- نائب مدير النظام
- المستخدمون الإداريون المركزيون

ما الذي يُدار من هذا المستوى؟

- إدارة المدارس
- إدارة حسابات المنصة
- مراجعة طلبات التسجيل
- إدارة المعلمين المستقلين
- إدارة الإعدادات العامة
- التحكم في سياسات النظام
- إدارة الأمان والجلسات
- إدارة التواصل المركزي والإشعارات العامة
- متابعة المؤشرات العامة
- إدارة الشروط والأحكام وسياسة الخصوصية
- مراقبة النشاط العام والتقارير المركزية

طبيعة هذا المستوى

هذا المستوى لا يدير الحصص اليومية داخل المدرسة مباشرة، ولكنه:

- يضبط الإطار العام
- يشرف على المدارس
- يراقب الأداء
- يدير السياسات
- يتحكم في البنية العامة للمنصة

ثانيًا: مستوى المدرسة

School Level

هذا هو المستوى التشغيلي الخاص بكل مدرسة على حدة.

من يعمل على هذا المستوى؟

- مدير المدرسة
- مساعد مدير المدرسة
- المعلم
- الطالب
- ولي الأمر
- المستخدمون الإداريون داخل المدرسة

ما الذي يُدار من هذا المستوى؟

- الطلاب
- المعلمون
- الفصول
- الصفوف
- أولياء الأمور
- ربط العلاقات بينهم
- الجداول الدراسية
- الحضور والانصراف
- التقييمات والاختبارات

- الإشعارات الداخلية
- التقارير المدرسية
- إعدادات المدرسة
- جاهزية البيانات للتشغيل

طبيعة هذا المستوى

هو المستوى الذي تحدث فيه العمليات اليومية الفعلية للمدرسة.

7) قاعدة جوهريّة: قاعدة البيانات هي المصدر الوحيد للحقيقة

هذه من أهم المبادئ في نَسَق:

جميع البيانات يجب أن تأتي من قاعدة البيانات

أي:

- عدد المدارس
- عدد الطلاب
- عدد المعلمين
- نسبة الحضور
- الجداول
- التنبيهات
- الرسائل
- الكروت
- الرسوم البيانية
- التقارير
- نتائج البحث
- حالات المستخدمين
- حالة الفصول
- حالة الجاهزية

كل ذلك يجب أن يكون ناتجًا من بيانات حقيقية مخزنة فعليًا.

ما الذي يمنع تمامًا؟

- استخدام أرقام ثابتة hardcoded داخل الواجهة
- استخدام demo data كحل نهائي
- استخدام mock content بدل الربط الحقيقي
- عرض نجاح مزيف لعمليات الحفظ
- إظهار بيانات غير موجودة في قاعدة البيانات
- الاعتماد على بيانات frontend-only

8) لا يوجد حفظ وهمي في الفرونت إند

هذا مبدأ تنفيذي صارم داخل نَسَق.

معنى ذلك

أي عملية:

- إنشاء
- تعديل
- حذف
- تفعيل
- تعليق
- ربط
- فك ربط
- جدولة
- إرسال
- تحديث إعدادات

يجب أن تتم End-to-End بالشكل التالي:

1. المستخدم ينفذ الإجراء من الواجهة
2. الواجهة ترسل الطلب للـ backend
3. الـ backend ينفذ المنطق
4. يتم الحفظ فعليًا في قاعدة البيانات
5. يتم تحديث الكيانات المرتبطة
6. تعود النتيجة للواجهة
7. تنعكس التغييرات في جميع الأماكن ذات العلاقة

أمثلة

إذا أضفت مدرسة جديدة:

- تُنشأ في قاعدة البيانات
- تظهر في إدارة المدارس
- تظهر في العدادات
- تظهر في إدارة المستخدمين إن كانت مرتبطة
- تدخل في التقارير
- تنعكس على مركز القيادة

إذا نقلت طالبًا من فصل إلى فصل:

- يتحدث ربط الطالب في DB
- يتغير عداد الفصلين
- تتحدث قائمة الطلاب
- تتحدث جاهزية الفصل
- تتأثر التقارير المرتبطة
- يتحدث أي Dashboard يعتمد على هذه البيانات

إذا عدلت مدة الحصة:

- لا يكفي أن يتغير النص في الواجهة
- يجب تحديث الإعدادات الفعلي
- يجب أن يعاد احتساب الجداول إن كان النظام يسمح
- يجب أن تتأثر التقارير والمؤشرات المرتبطة

9) النظام ديناميكي بالكامل

نَسَقُ يجب أن يكون Very Dynamic.

أي أن النظام:

- لا ينهار إذا لم توجد بيانات
- لا يحتاج إلى Demo Data كي يبدو مكتملاً
- يعرض Empty States حقيقية
- يعتمد على علاقات الكيانات
- يحدث نفسه بناءً على البيانات الفعلية
- يُظهر الحقيقة كما هي، لا كما يراد تجميلها

أمثلة على السلوك الديناميكي

- إذا لم توجد مدارس: يظهر Empty State واضح
- إذا لم يوجد طلاب: لا تعرض المنصة طلاباً وهميين
- إذا لم يوجد حضور كافٍ: لا تعرض نسباً مزروعة
- إذا لم تُنشأ جداول: لا تعرض جداول مرسومة يدوياً
- إذا لم توجد إشعارات: لا تُظهر نصوصاً تجريبية

10) الأدوار الرئيسية داخل النظام

يجب فهم نَسَقُ على أنه نظام متعدد الأدوار.

1. مدير النظام

يدير المنصة بالكامل

2. نائب مدير النظام

يساعد في الإدارة المركزية

3. مدير المدرسة

يدير مدرسة واحدة

4. مساعد مدير المدرسة

يدعم التشغيل المدرسي

5. المعلم

يدير ما يخص تدريسه وفصوله وحضوره وتقييماته

6. الطالب

يتابع جدولته وحضوره ونتائجه إن كان الدور مفتلاً

7. ولي الأمر

يتابع أبناءه وبياناتهم

8. المعلم المستقل

يتقدم من التسجيل العام وتتم مراجعته مركزياً

11) الكيانات الأساسية داخل النظام

لفهم النظام، يجب فهم الكيانات الأساسية التي يقوم عليها، ومنها:

- Platform
- Schools
- Users
- User Identity
- Roles
- Permissions
- Academic Years
- Grade Levels
- Classes
- Students
- Teachers
- Parents
- Student-Parent Relations
- Teacher-Class Assignments
- Teacher-Subject Assignments
- Subjects
- Schedules
- Schedule Entries
- Attendance Records
- Assessments
- Grades
- Notifications
- Messages
- Reports
- Audit Logs
- Security Logs
- Registration Requests
- Settings
- AI Insights

كل هذه الكيانات يجب أن تكون مترابطة بمنطق واضح.

12) المحركات الأساسية للنظام

نَشَق لا يعمل كصفحات مستقلة، بل كمنظومة مبنية على محركات تشغيلية أساسية.

Identity Engine .1

مسؤول عن:

- الهوية الموحدة
- المستخدمين
- الأدوار
- الصلاحيات
- العلاقات بين المستخدمين

Scheduling Engine .2

مسؤول عن:

- الجداول الدراسية
- توزيع المواد
- ربط المعلمين بالفصول
- توزيع الحصص
- منع التعارضات

Attendance Engine .3

مسؤول عن:

- حضور الطلاب
- حضور المعلمين
- ربط الحضور بالحصّة
- نسب الحضور
- التقارير المرتبطة

Assessment Engine .4

مسؤول عن:

- الاختبارات
- التقييمات
- الدرجات
- التحليل الأكاديمي

Notification Engine .5

مسؤول عن:

- الإشعارات
- الرسائل
- التنبيهات
- الجدولة
- التوزيع على الفئات

AI Insight Engine .6

مسؤول عن:

- الرؤية الذكية
- التحليل
- الإنذار المبكر
- التوصيات
- دعم القرار

7. Reporting Engine

مسؤول عن:

- التقارير
- المؤشرات
- dashboards
- charts
- التحليل التجميعي

8. Tenant Engine

مسؤول عن:

- إدارة المدارس
- عزل البيانات
- إعدادات كل مدرسة
- السياق المدرسي

13) كيف يجب أن يفهم الـ Agent النظام؟

إذا كان سيتم تسليم هذه الوثيقة إلى AI Agent جديد، فيجب أن يفهم بوضوح أن:

نَسَق ليس:

- Prototype
- UI Demo
- Dashboard شكلي
- مجموعة صفحات عرض فقط

نَسَق هو:

- نظام تشغيل مدرسي حقيقي
- منصة متعددة المدارس
- نظام متعدد الأدوار
- منصة تعتمد على هوية موحدة
- نظام يعتمد على قاعدة بيانات حقيقية
- نظام لا يقبل fake save
- نظام لا يقبل mock content كحل نهائي
- نظام يجب أن يعمل end-to-end

14) ما الذي يجب على الـ Agent احترامه أثناء البناء أو الاستكمال؟

أي Agent أو مطور جديد يعمل على نَسَق يجب أن يلتزم بالتالي:

- الحفاظ على Multi-Tenant Architecture
- الحفاظ على Unified User Identity
- الحفاظ على الفصل الواضح بين Platform Level و School Level
- عدم استخدام قيم hardcoded
- عدم استخدام mock data كحل نهائي
- عدم تنفيذ حفظ وهمي في الواجهة
- عدم إنشاء صفحات مكررة أو قديمة
- عدم كسر العلاقات بين الكيانات
- عدم إظهار بيانات تجريبية بدل البيانات الحقيقية
- جعل كل شيء يعتمد على قاعدة البيانات
- جعل كل تغيير ينعكس على جميع الأجزاء المرتبطة

15) السلوك المتوقع من النظام ككل

النظام يجب أن يعمل كبيئة مترابطة، بحيث:

- إنشاء مدرسة ينعكس في كل الأماكن المرتبطة
- إنشاء معلم ينعكس في إدارة المستخدمين والجداول والحضور
- إنشاء طالب ينعكس في الفصل وولي الأمر والتقارير
- تعديل فصل ينعكس في الطلاب والمعلمين والجداول
- تغيير إعداد عام ينعكس على النظام كله
- تغيير إعداد مدرسة ينعكس على سياقها فقط
- تسجيل حضور ينعكس في النسب والعدادات والتقارير
- إرسال إشعار ينعكس في السجلات والمستلمين
- أي عملية حساسة تُسجل في Audit Logs عند الحاجة

16) خلاصة الوثيقة

نَسَق | NASSAQ هو School Operating System متعدد المدارس، متعدد الأدوار، قائم على Unified User Identity، ويعمل من خلال مستويين رئيسيين هما Platform Level و School Level. كل بياناته يجب أن تأتي من قاعدة البيانات الفعلية، وكل عملية داخله يجب أن تكون حقيقية End-to-End دون أي حفظ وهمي في الفرونت إند.

النظام مبني ليكون:

- قابلاً للتوسع
- قابلاً للإدارة المركزية
- قابلاً لتشغيل المدرسي اليومي
- قائماً على كيانات مترابطة
- مدعوماً بمحركات تشغيلية واضحة
- وقادراً على تقديم تجربة تشغيل تعليمية حقيقية وليست شكلية